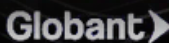


PROYECTO FINAL - INTEGRANDO TODO LO DADO

Python Django & MySql

Damos inicio al final

En este proyecto final vamos a integrar todos los conocimientos adquiridos a lo largo del curso para confeccionar una web final usando DJANGO como framework



INFORMATARIO



Subsecretaría de
Empleo

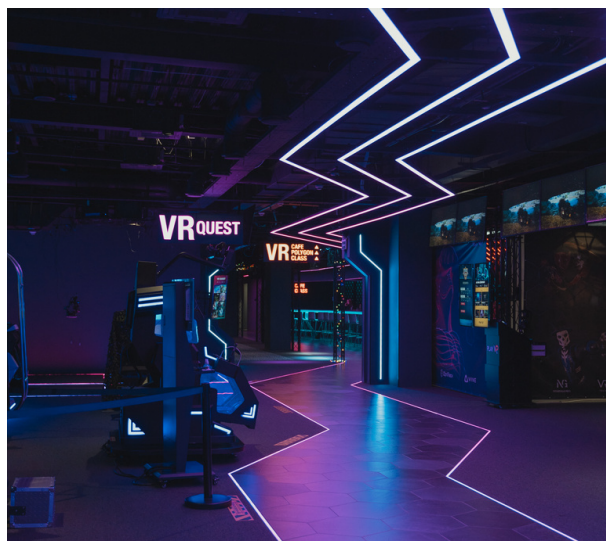


Ministerio de
Producción, Industria y Empleo



CHACO
Gobierno de todos

CREANDO EL ENTORNO VIRTUAL



>>> ¿QUÉ ES EL ENTORNO VIRTUAL?

Imagina que estás trabajando en tres proyectos diferentes en un taller. En el taller, tienes 50 herramientas diferentes que podrías necesitar para los proyectos, como destornilladores, martillos, llaves inglesas, etc.

Si el taller está desordenado y todas las herramientas están mezcladas, sería difícil encontrar las herramientas específicas que necesitas para cada proyecto. Tendrías que buscar en toda la sala y perderías tiempo valioso tratando de ubicar las herramientas correctas.

Pero, si el taller está organizado en sectores, donde cada sector corresponde a un proyecto en particular, y las herramientas necesarias para cada proyecto se colocan en su respectivo sector, la situación mejora. Cada sector del taller tendría las herramientas específicas que necesitas para ese proyecto en particular.

Además, hay un sector neutro donde se colocan las herramientas que se comparten entre los proyectos. Estas herramientas son utilizadas por varios proyectos y se mantienen en un lugar central para que todos los proyectos puedan acceder fácilmente a ellas.

Con este enfoque, puedes ir al sector correspondiente al proyecto en el que estás trabajando y encontrar todas las herramientas necesarias allí mismo, sin tener que buscar en otros lugares. También puedes consultar el sector neutro cuando necesites herramientas compartidas.

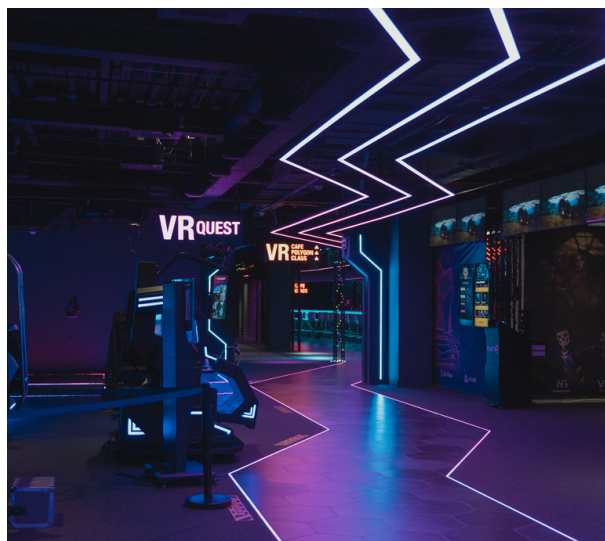


En el desarrollo de software, los proyectos serían tus diferentes aplicaciones o sistemas, y las herramientas serían las bibliotecas, paquetes y configuraciones específicas para cada proyecto. Un entorno virtual sería como tener sectores ordenados en el taller, donde cada sector representa un proyecto y contiene las herramientas necesarias para ese proyecto. El sector neutro sería utilizado para herramientas compartidas que son utilizadas por varios proyectos.

De esta manera, los entornos virtuales te permiten trabajar en varios proyectos de manera organizada y eficiente, asegurándote de tener todas las herramientas necesarias disponibles sin confusiones ni conflictos entre los proyectos.

Entonces podemos decir que, un entorno virtual es como tener un taller organizado en sectores, donde cada sector representa un proyecto y contiene las herramientas específicas para ese proyecto. Esto facilita el acceso a las herramientas correctas sin tener que buscar en todas partes, y también proporciona un espacio central para las herramientas compartidas.

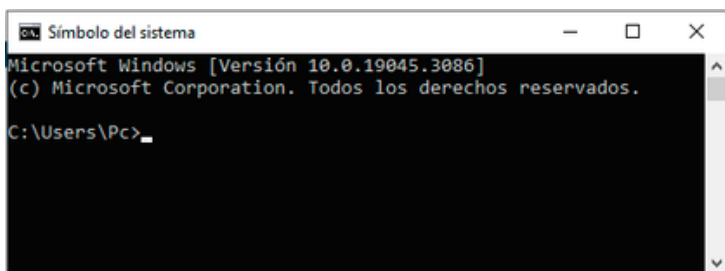
PASOS PARA LA CREACIÓN DEL ENTORNO



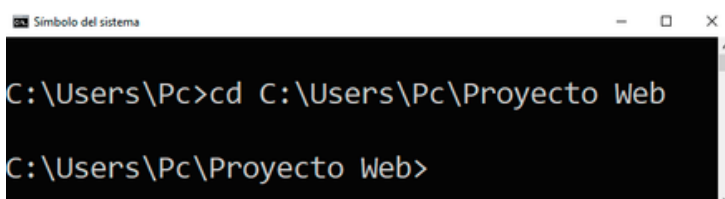
>>> PASOS

Ahora que ya sabes lo que es un entorno virtual, vamos a crearlo (ya lo habrás visto en clase, pero te lo dejamos plasmado desde aquí):

1 - Trabajando desde Windows, vamos a abrir una consola (CMD).



Lo que debes hacer ahora es ubicarte en la carpeta donde vas a crear este proyecto final. Esto lo puedes hacer usando el comando `cd`, seguido de la ubicación donde estará tu proyecto final, o en este caso, donde vas a crear tu entorno virtual.

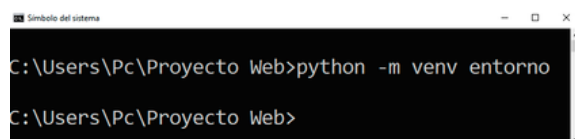


Utiliza el siguiente comando para instalar la paquetería necesaria para crear el entorno:

- **`pip install virtualenv`** (este comando te servirá para instalar la paquetería necesaria para crear tus entornos virtuales).

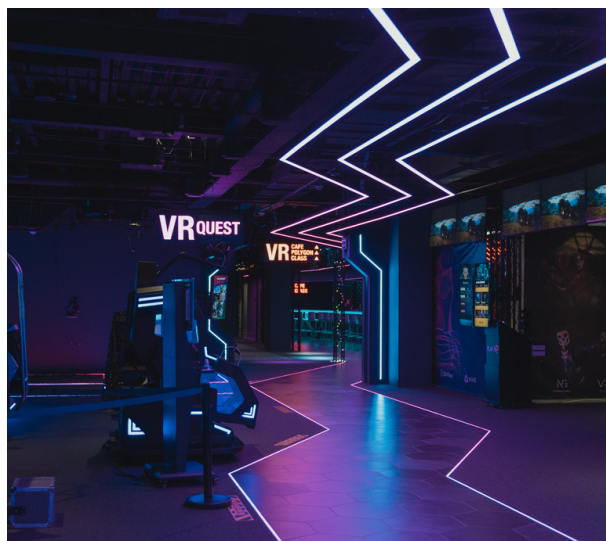
Procedemos a la creación del entorno:

- **`python -m venv entorno`** (este comando te servirá para crear tu entorno virtual, donde "entorno" es el nombre que nosotros escogimos para el entorno, puedes escoger cualquier nombre).



Ya tenemos nuestro entorno creado. Siguiendo en la misma ubicación, vamos a movernos a la carpeta que se creó que es la de nuestro entorno, utilizando el comando `cd` seguido del nombre del entorno creado:

PASOS PARA INSTALAR DJANGO



»»» PASOS

```
Simbolo del sistema
C:\Users\Pc\Proyecto Web>cd entorno
C:\Users\Pc\Proyecto Web\entorno>
```

Si lo miras a través del explorador de Windows, podrás ver que esta carpeta se creó a partir del comando que utilizamos para crear el entorno. Dentro de esta carpeta se encuentran los archivos necesarios para nuestro entorno, sin embargo, para instalar Django (y el resto de paquetes que hagan falta), debemos activar nuestro entorno. Para ello usamos el comando `cd` nuevamente y nos dirigimos a la carpeta **Scripts**:

```
Simbolo del sistema
C:\Users\Pc\Proyecto Web\entorno>cd Scripts
C:\Users\Pc\Proyecto Web\entorno\Scripts>
```

Dentro de esta carpeta procedemos a activar nuestro entorno para poder comenzar con la instalación de nuestros paquetes y comenzar a trabajar en el proyecto final. Usamos el comando **activate**:

```
Simbolo del sistema
C:\Users\Pc\Proyecto Web\entorno\Scripts>activate
(entorno) C:\Users\Pc\Proyecto Web\entorno\Scripts>
```



Notarás como al principio de la línea de comando ahora se encuentra entre paréntesis, el nombre de nuestro entorno. Esto quiere decir que nuestro entorno está activado y listo para comenzar a trabajar en él. A modo de ordenar mejor todo, vamos a movernos a la carpeta raíz utilizando el comando `cd..` (palabra `cd` y dos puntos). Lo haremos 2 veces para llegar a la carpeta principal:

```
Simbolo del sistema
C:\Users\Pc\Proyecto Web\entorno\Scripts>activate
(entorno) C:\Users\Pc\Proyecto Web\entorno\Scripts>cd..
(entorno) C:\Users\Pc\Proyecto Web\entorno>cd..
(entorno) C:\Users\Pc\Proyecto Web>
```

Llega el momento de instalar Django por lo que usamos el comando **pip install django**:

COMENZAMOS NUESTRO PROYECTO

```

this, a(window).on("load", function() {
    use strict";function b(b){return this.each(function(){var
function(b){this.element=a(b);c.VERSION="3.3.7",c.TRANSITION_DURATION=1
b.data("target");if(d||(d=b.attr("href"),d=d&&.replace(/.*(?=[^\s]*$
{"hide.bs.tab",{relatedTarget:b[0]}},g=a.Event("show.bs.tab",{relatedTar
{var h=a(d);this.activate(b.closest("li"),c),this.activate(h,h.parent()
own.bs.tab",relatedTarget:e[0]}))}}},c.prototype.activate=function(b,d
moveClass("active").end().find("[data-toggle="tab"]').attr("aria-expanded
,h?(b[0].offsetWidth,b.addClass("in")):b.removeClass("fade"),b.parent(
oggle="tab"]').attr("aria-expanded",!0),e&&e())var g=d.find("> .active
.fade").length);g.length&&h?g.one("bsTransitionEnd",f).emulateTransiti
a.fn.tab=b,a.fn.tab.Constructor=c,a.fn.tab.noConflict=function(){return
ion b(b){return this.each(function(){var d=a(this),e=d.data("bs.affix
)))var c=function(b,d){this.options=a.extend({},c.DEFAULTS,d),this.$
checkPosition(this)).on("click.bs.affix.data-api",a.proxy(this.checkPosi
Offset=null,this.checkPosition());c.VERSION="3.3.7",c.RESET="affix af
a,b,c,d){var e=this.$target.scrollTop(),f=this.$element.offset(),g=th
:null!=d&&i+j">a-d&&"bottom"},c.prototype.getPinnedOffset=function()
s("affix");var a=this.$target.scrollTop(),b=this.$element.offset();r
function(){setInterval(a.proxy(this.checkPosition,this),100);this.options.offset,e=d.top,f=d.bottom
his.options.offset,e=d.top,f=d.bottom
this.$element)}

```

>>> PASOS

```

(entorno) C:\Users\Pc\Proyecto Web>pip install django
Collecting django
  Using cached Django-4.2.2-py3-none-any.whl (8.0 MB)
Collecting asgiref<4,>=3.6.0
  Using cached asgiref-3.7.2-py3-none-any.whl (24 kB)
Collecting sqlparse>=0.3.1
  Using cached sqlparse-0.4.4-py3-none-any.whl (41 kB)
Collecting tzdata
  Using cached tzdata-2023.3-py2.py3-none-any.whl (341 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.7.2 django-4.2.2 sqlparse-0.4.4 tzdata-2023.3

[notice] A new release of pip available: 22.3.1 -> 23.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip

(entorno) C:\Users\Pc\Proyecto Web>

```

Django ya se encuentra instalado en nuestro entorno y podremos comenzar a trabajar con él.

*** si te aparecen esos mensajes de "notice" al final de la instalación de Django, no te preocupes, solo se está avisando que la versión de **pip** que estamos utilizando es una versión anterior a la actual, y nos dice que si queremos actualizar a la nueva versión debemos usar los comandos escritos en verde.

Lo puedes actualizar o seguirlo usando como lo vienes haciendo hasta ahora.***

Ya estamos listos para comenzar nuestro proyecto final y para eso vamos a utilizar el comando:



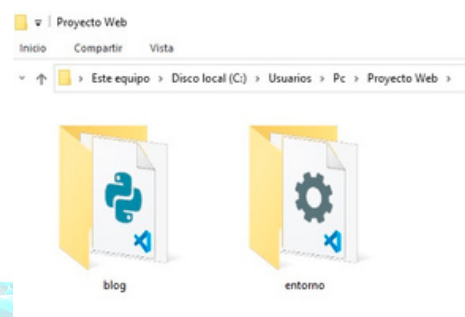
django-admin startproject blog (en este caso usamos el nombre "blog" para nuestro proyecto, pero puedes usar el nombre que quieras):

```

(entorno) C:\Users\Pc\Proyecto Web>django-admin startproject blog
(entorno) C:\Users\Pc\Proyecto Web>

```

Hecho esto, ya está creado nuestro primer proyecto y ahora si estamos listos para comenzar a trabajar. Si abres un explorador de Windows podrás visualizar mejor la estructura que tenemos, en la cual tenemos la carpeta "entorno" que creamos a partir del comando **pip -m venv entorno** y ahora la carpeta "blog" a partir del comando para iniciar nuestro proyecto **django-admin startproject blog**



ESTRUCTURANDO NUESTRO PROYECTO

```

, this, a(window).on("load", function() {
    "use strict";
    function b(b){return this.each(function(){var
    function(b){this.element=a(b)};c.VERSION="3.3.7",c.TRANSITION_DURATION=1
    b.data("target");if(d||(d=b.attr("href"),d=d&&d.replace(/.*(?=#[^\s]*$
    ("hide.bs.tab",{relatedTarget:b[0]}),g=a.Event("show.bs.tab",{relatedTar
    (var h=a(d);this.activate(b.closest("li"),c),this.activate(h,h.parent()
    own.bs.tab",{relatedTarget:e[0]}))}}},c.prototype.activate=function(b,d
    moveClass("active").end().find("[data-toggle="tab"]').attr("aria-expand
    ,h?[b[0].offsetWidth,b.addClass("in")):b.removeClass("fade"),b.parent(
    gggle="tab"]').attr("aria-expanded",!0),e&&e())var g=d.find("> .active
    .fade").length;g.length&&h?g.one("bsTransitionEnd",f).emulateTransiti
    a.fn.tab=b,a.fn.tab.Constructor=c,a.fn.tab.noConflict=function(){return
    ent).on("click.bs.tab.data-api",'[data-toggle="tab"]',e).on("click.bs.
    ion b(b){return this.each(function(){var d=a(this),e=d.data("bs.affix
    )))var c=function(b,d){this.options=a.extend({},c.DEFAULTS,d),this.$
    checkPosition(this)).on("click.bs.affix.data-api",a.proxy(this.checkP
    Offset=null,this.checkPosition());c.VERSION="3.3.7",c.RESET="affix af
    a,b,c,d)(var e=this.$target.scrollTop(),f=this.$element.offset(),g=th
    :null!=d&i+j)=a-d&&"bottom"},c.prototype.getPinnedOffset=function()
    s("affix");var a=this.$target.scrollTop(),b=this.$element.offset();r
    function(){setTimeOut(a.proxy(this.checkPosition,this),1)};this.options.offset=e,d.top=f,d.bott
    this.options.offset,e=d.top,f=d.bott
    p(this.$element)};
    
```

»»» PASOS

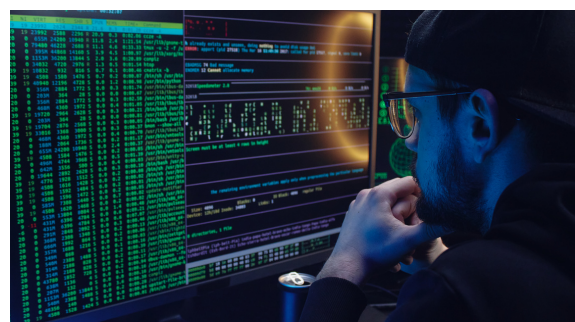
Si accedemos a la carpeta blog que se creó para nuestro proyecto, vamos a ver tenemos un archivo **manage.py** y otra carpeta llamada **blog**.

El archivo **manage.py** va a ser uno de los archivos fundamentales que vamos a usar ya que con el podremos correr nuestro servidor, entre otras muchas funciones que nos brindará.

correr el servidor: nos referimos a que se generará un servidor local web con el cual vamos a poder ver el progreso de nuestra página a medida que la vamos modificando, como si estuviéramos navegando por internet, pero en realidad solo vamos a estar trabajando de manera local en nuestra pc, es decir, si durante este tiempo (hasta la instalación de otros paquetes) no tienes conexión a internet, de todas formas podrás ver tu página web y seguir trabajando en ella.

Si accedemos a la carpeta "blog" veremos que se encuentran otros archivos dentro de ella.

- El archivo **__init__.py** es un archivo especial en Python que se utiliza para indicar que un directorio es un paquete de Python. Aunque el archivo puede estar vacío, su presencia es esencial para que Python reconozca el directorio como un paquete válido.



- El archivo **asgi.py** es un archivo que se encuentra en un proyecto Django y se utiliza para configurar y ejecutar el servidor ASGI (Asynchronous Server Gateway Interface). ASGI es una especificación que permite que las aplicaciones web en Python se comuniquen con servidores web de manera asíncrona, lo que las hace adecuadas para manejar solicitudes concurrentes en tiempo real.
- El archivo **settings.py** es un archivo importante en un proyecto Django. Contiene la configuración principal de tu aplicación, como la base de datos, la configuración de aplicaciones, las claves secretas, las rutas de archivos estáticos y más. Esencialmente, el archivo **settings.py** controla el comportamiento y la apariencia de tu proyecto Django.

ESTRUCTURANDO NUESTRO PROYECTO

```

    this, a(window).on("load", function() {
        use strict";function b(b){return this.each(function(){var
        function(b){this.element=a(b)};c.VERSION="3.3.7", c.TRANSITION_DURATION=1
        b.data("target");if(d||(d=b.attr("href"),d=d&&d.replace(/.*(?=#[^\s]*$
        ("hide.bs.tab",{relatedTarget:b[0]}),g=a.Event("show.bs.tab",{relatedTar
        (var h=a(d);this.activate(b.closest("li"),c),this.activate(h,h.parent()
        own.bs.tab",{relatedTarget:e[0]}))}}},c.prototype.activate=function(b,d
        moveClass("active").end().find("[data-toggle="tab"]').attr("aria-expanded
        ,h?[0].offsetWidth,b.addClass("in")):b.removeClass("fade"),b.parent(
        gggle="tab"]').attr("aria-expanded",!0),e&&e())var g=d.find("> .active
        .fade").length;g.length&&h?g.one("bsTransitionEnd",f).emulateTransiti
        a.fn.tab=b,a.fn.tab.Constructor=c,a.fn.tab.noConflict=function(){return
        ent).on("click.bs.tab.data-api",[data-toggle="tab"],e).on("click.bs.
        ion b(b){return this.each(function(){var d=a(this),e=d.data("bs.affix
        )))var c=function(b,d){this.options=a.extend({},c.DEFAULTS,d),this.$
        checkPosition(this)).on("click.bs.affix.data-api",a.proxy(this.checkP
        Offset=null,this.checkPosition());c.VERSION="3.3.7",c.RESET="affix af
        a,b,c,d)(var e=this.$target.scrollTop(),f=this.$element.offset(),g=th
        :null!-d&&i+j>a-d&&"bottom"},c.prototype.getPinnedOffset=function()
        s("affix");var a=this.$target.scrollTop(),b=this.$element.offset();r
        function(){setTimeout(a.proxy(this.checkPosition,this),100)};this.
        this.options.offset,e=d.top,f=d.bottom
        (this.$element)}
    });

```

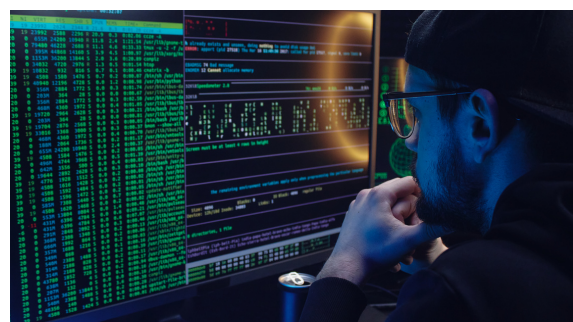
»»» PASOS

- El archivo **urls.py** es un archivo de configuración clave en un proyecto Django. Define las rutas de URL de tu aplicación y cómo se relacionan con las vistas correspondientes. En otras palabras, el archivo `urls.py` mapea las URL que los usuarios ingresan en su navegador a las funciones o clases de vistas que deben manejar esas solicitudes.
- El archivo **wsgi.py** es un archivo de configuración utilizado en proyectos Django que implementan el estándar WSGI (Web Server Gateway Interface). WSGI es una especificación que define cómo las aplicaciones web en Python se comunican con los servidores web.

Vamos comenzar nuestro proyecto para trabajar de forma más ordenada, sin embargo, debemos aclarar que esto es solo una manera de hacerlo y lo hacemos por convención; si quieres realizarlo de otra manera, puedes hacerlo, pero, a fines prácticos para este proyecto, te recomendamos seguir la estructura que usaremos para que veas el paso a paso que iremos haciendo y puedas seguirnos de la misma forma.

Vamos a posicionarnos en la carpeta "blog" donde encontramos el archivo `manage.py` y la subcarpeta "blog".

Una vez posicionados ahí, vamos a crear una serie de



carpetas que utilizaremos durante el proyecto.

La creación de éstas carpetas las puedes hacer desde el explorador de Windows o desde la consola. Nosotros utilizaremos la consola para crearlas, y utilizaremos para esto los comandos **dir** y **mkdir**:

Con **dir** podemos verificar las carpetas y archivos que se encuentran dentro de la carpeta actual en la que estamos y con el comando **mkdir** vamos a crear las carpetas.

```

(entorno) C:\Users\Pc\Proyecto Web\blog>dir
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: 6C77-C982

Directorio de C:\Users\Pc\Proyecto Web\blog

<DIR>          ..
<DIR>          .
<DIR>          blog
                682 manage.py
                682 bytes
                1 archivos
                3 dirs  29.880.606.720 bytes libres

(entorno) C:\Users\Pc\Proyecto Web\blog>

```

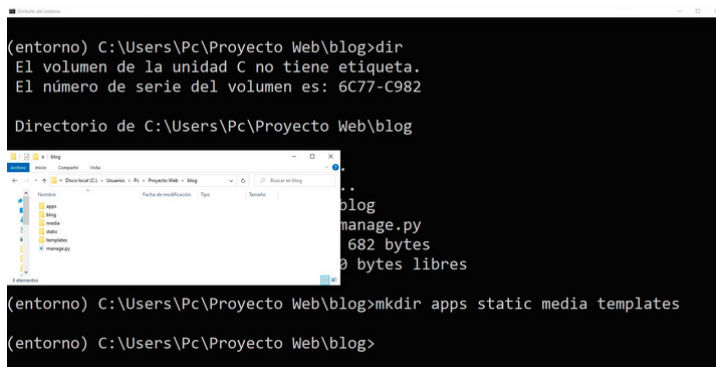
ESTRUCTURANDO NUESTRO PROYECTO

```

this, a(window).on("load", function() {
    use strict"; function b(b){return this.each(function() {
    function(b){this.element=a(b);c.VERSION="3.3.7", c.TRANSITION_DURATION=1
    b.data("target");if(d||(d=b.attr("href"),d=d&&d.replace(/.*(?=[^\s]*$/
    ("hide.bs.tab",{relatedTarget:b[0]}),g=a.Event("show.bs.tab",{relatedTar
    (var h=a(d);this.activate(b.closest("li"),c),this.activate(h,h.parent()
    own.bs.tab",relatedTarget:e[0]}))));c.prototype.activate=function(b,d
    moveClass("active").end().find("[data-toggle="tab"]').attr("aria-expand
    ,h?(b[0].offsetWidth,b.addClass("in")):b.removeClass("fade"),b.parent(
    oggle="tab"]').attr("aria-expanded",!0),e&&e())var g=d.find("> .active
    .fade").length;g.length&&h?g.one("bsTransitionEnd",f).emulateTransiti
    a.fn.tab=b,a.fn.tab.Constructor=c,a.fn.tab.noConflict=function(){return
    ent).on("click.bs.tab.data-api",'[data-toggle="tab"]',e).on("click.bs.
    ion b(b){return this.each(function() {var d=a(this),e=d.data("bs.affix
    )))var c=function(b,d){this.options=a.extend({},c.DEFAULTS,d),this.$
    checkPosition(this)).on("click.bs.affix.data-api",a.proxy(this.checkP
    Offset=null,this.checkPosition());c.VERSION="3.3.7",c.RESET="affix af
    a,b,c,d){var e=this.$target.scrollTop(),f=this.$element.offset(),g=th
    :null!-d&&i+j>a-d&&"bottom"},c.prototype.getPinnedOffset=function()
    s("affix");var a=this.$target.scrollTop(),b=this.$element.offset();r
    function(){setTimeOut(a.proxy(this.checkPosition,this),1)}this.options
    this.options.offset,e=d.top,f=d.bottom
    (this.$element)}
    
```

»»» PASOS

Vamos a crear una carpeta llamada "apps", luego una carpeta llamada "static", otra llamada "media" y otra "templates".



```

(entorno) C:\Users\Pc\Proyecto Web\blog>dir
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: 6C77-C982

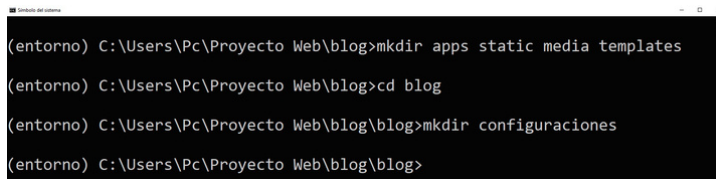
Directorio de C:\Users\Pc\Proyecto Web\blog

.                <-->
..               <-->
blog
manage.py
682 bytes
0 bytes libres

(entorno) C:\Users\Pc\Proyecto Web\blog>mkdir apps static media templates
(entorno) C:\Users\Pc\Proyecto Web\blog>
    
```

Puedes ver que con el comando **mkdir** y luego los nombres de las carpetas separadas por espacio, se crean todas las carpetas de una vez. En esta imagen puedes ver el comando y anexamos una captura de pantalla del explorador de Windows para que se pueda apreciar como se crearon las carpetas.

Ahora vamos a posicionarnos en la subcarpeta "blog" donde crearemos una carpeta llamada "configuraciones".



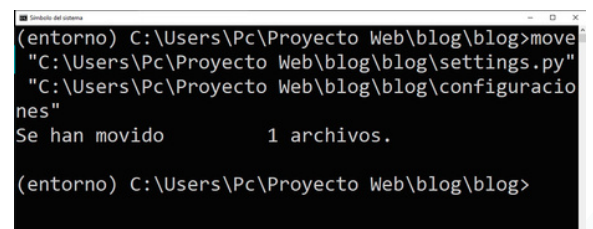
```

(entorno) C:\Users\Pc\Proyecto Web\blog>mkdir apps static media templates
(entorno) C:\Users\Pc\Proyecto Web\blog>cd blog
(entorno) C:\Users\Pc\Proyecto Web\blog\blog>mkdir configuraciones
(entorno) C:\Users\Pc\Proyecto Web\blog\blog>
    
```



Si lo miramos desde el explorador de Windows podremos ver como a quedado nuestra estructura, o desde cmd, usando el comando **dir**.

Estando en esta carpeta vamos a mover el archivo settings.py a la carpeta configuraciones que acabamos de crear (puedes hacerlo desde la consola tal cual te mostraremos con el comando **move** o desde el explorador de Windows cortando y pegando el archivo a la carpeta "configuraciones").



```

(entorno) C:\Users\Pc\Proyecto Web\blog\blog>move
"C:\Users\Pc\Proyecto Web\blog\blog\settings.py"
"C:\Users\Pc\Proyecto Web\blog\blog\configuracio
nes"
Se han movido          1 archivos.

(entorno) C:\Users\Pc\Proyecto Web\blog\blog>
    
```

CREANDO LOS ARCHIVOS LOCAL.PY Y PROD.PY

```

        this, a(window).on("load", function() {
            "use strict";
            function b(b){return this.each(function(){var c=
            function(b){this.element=a(b)};c.VERSION="3.3.7", c.TRANSITION_DURATION=1
            b.data("target");if(d||(d=b.attr("href"),d=d&&d.replace(/.*(?=#[^\s]*$
            {"hide.bs.tab",{relatedTarget:b[0]}},g=a.Event("show.bs.tab",{relatedTar
            {"var h=a(d);this.activate(b.closest("li"),c),this.activate(h,h.parent()
            {"own.bs.tab",relatedTarget:e[0]}))}}},c.prototype.activate=function(b,d
            moveClass("active").end().find("[data-toggle="tab"]').attr("aria-expanded
            ,h?(b[0].offsetWidth,b.addClass("in")):b.removeClass("fade"),b.parent(
            oggle="tab"]').attr("aria-expanded",!0),e&&e())var g=d.find("> .active
            .fade").length;g.length&&h?g.one("bsTransitionEnd",f).emulateTransiti
            a.fn.tab=b,a.fn.tab.Constructor=c,a.fn.tab.noConflict=function(){return
            ent).on("click.bs.tab.data-api",[data-toggle="tab"],e).on("click.bs.
            ion b(b){return this.each(function(){var d=a(this),e=d.data("bs.affix
            )))var c=function(b,d){this.options=a.extend({},c.DEFAULTS,d),this.$
            checkPosition(this)).on("click.bs.affix.data-api",a.proxy(this.checkP
            Offset=null,this.checkPosition());c.VERSION="3.3.7",c.RESET="affix af
            a,b,c,d){var e=this.$target.scrollTop(),f=this.$element.offset(),g=th
            :null!-d&&i+j>=a-d&&"bottom"},c.prototype.getPinnedOffset=function()
            s("affix");var a=this.$target.scrollTop(),b=this.$element.offset();r
            function(){setInterval(a.proxy(this.checkPosition,this),100)};this.
            this.options.offset,e=d.top,f=d.bottom
            p(this.$element)};
    
```

>>> PASOS

Como puedes ver, usamos el comando **move**, luego la carpeta de origen y la carpeta de destino y, usamos comillas para poner la ruta de cada carpeta.

Hecha esta estructuración, podemos comenzar a trabajar con nuestro editor de código para realizar las primeras configuraciones y poner a funcionar nuestro servidor. Ya puedes cerrar la consola cmd que tienes abierta, ya que ahora trabajaremos con la consola cmd pero en VSC.

Abrimos Visual Studio Code y elegimos la carpeta de nuestro proyecto. Una vez ahí, vamos a ir a la carpeta de configuraciones y vamos a crear dos archivos nuevos: **local.py** y **prod.py** y, dentro del archivo **local.py** vamos a importar todo lo que haya en el archivo **settings.py**.



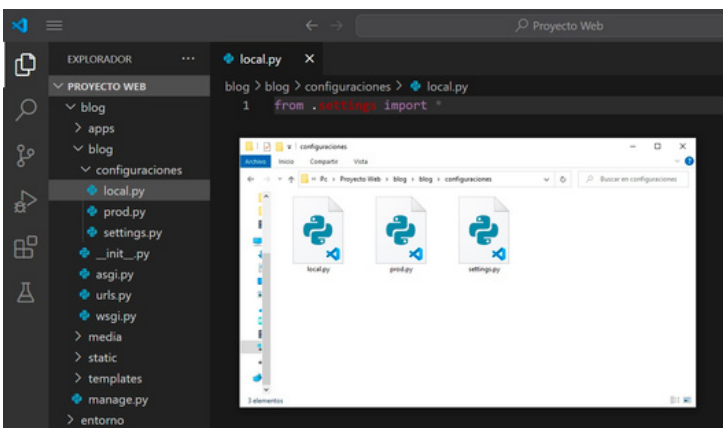
En esta captura de pantalla anexamos el explorador de Windows para que puedas ver como están creados los archivos (además de verlos en el explorador de VSC).

Luego abrimos el archivo **local.py** y en la primer línea de comandos escribimos **from .settings** **import *** para traer todas las configuraciones del archivo **settings.py** a nuestro archivo **local.py**.

Siempre que escribamos una línea de comando, debemos guardar el archivo, y una de las formas más rápidas es presionando la tecla **ctrl + s**.

Ya verás, más adelante el motivo de la creación de estos archivos, pero por el momento, si estás siguiendo nuestros pasos, podrás seguir sin problemas.

Por último, antes de correr nuestro servidor y verificar que nuestro proyecto está funcionando correctamente, vamos a realizar unas configuraciones más.



MODIFICANDO EL ARCHIVO SETTINGS.PY

```

this),a(window).on("click.bs.tab",function(b){return this.each(function(){var c=
"use strict";function b(b){return this.each(function(){c.VERSION="3.3.7",c.TRANSITION_DURATION=1
function(b){this.element=a(b);c.VERSION="3.3.7",c.TRANSITION_DURATION=1
b.data("target");if(d[0]&&(d=b.attr("href"),d=d&&replace(/.*\?=#[\^$%&
,"hide.bs.tab",{relatedTarget:b[0]}),g=a.Event("show.bs.tab",{relatedTa
({var h=a(d),this.activate(b.closest("li"),c),this.activate(h,h.parent(
own.bs.tab",{relatedTarget:e[0]}))}}),c.prototype.activate=function(b,c
moveClass("active").end().find("[data-toggle="tab"]').attr("aria-expand
,h[b[0].offsetWidth,b.addClass("in")],g.a.removeClass("fade"),b.parent(
ogle="tab").attr("aria-expanded",!0),e&&e())var g=d.find(">.active
c.length);g.length&&g.one("bsTransitionEnd",f).emulateTransi
a.fn.tab&&a.fn.tab.Constructor=c,a.fn.tab.noConflict=function(){return
ent).on("click.bs.tab.data-api",[data-toggle="tab"],e).on("click.bs
))var b(b){return this.each(function(){var d=a(this),e=d.data("bs.affix
))var c=function(b,d){this.options=a.extend({},c.DEFAULTS,d),this.$
Offset=null,this.checkPosition();},a.proxy(this.checkP
a,b,d){var e=this.$target.scrollTop(),c.VERSION="3.3.7",c.RESET="affix a
(affixed)return null==c?(+this.unpin<f.top)&&"bottom":!(+g<a-d)&&
s("affix");var a=d&&"bottom",c.prototype.getPinnedOffset=function(X
ction(){$.setTimeout(a.proxy(this.checkPosition(),b,this.$element.offset(),
this.options.offset,e,d,top,f,d,bottom),this.$element.offset());
to(this.$element.offset(),e,d,top,f,d,bottom),this.$element.offset());

```

➤➤➤ PASOS

Vamos a abrir el archivo **settings.py** para modificar ciertos puntos iniciales de modo tal que los siguientes pasos, sean más fluídos.

Vamos a guiarte por el número de línea que debes modificar dentro del archivo `settings.py` (no te preocupes que a medida que necesitemos, iremos explicándote las partes del código de este archivo) para que sea más fácil seguir los pasos ahora.

- En la línea de código 12, importaremos el módulo **os** que nos ayudará a interactuar con nuestro sistema operativo (gestionar archivos, directorios, variables de entorno, entre otras cosas).

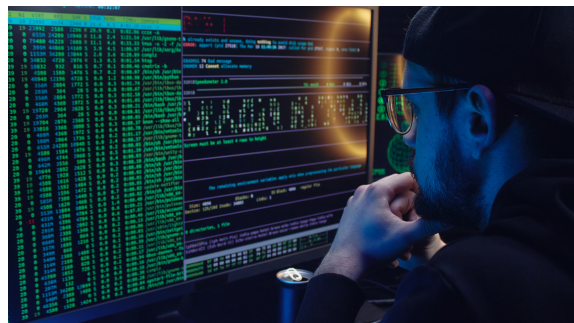
```
12 import os
13 from pathlib import Path
```

- En la línea de código 57 vamos a configurar el acceso a nuestra carpeta "templates" (desde donde Django accederá a nuestras plantillas html para dar respuesta a las solicitudes HTTP).

```
56     'BACKEND': 'django.template.backends.django.DjangoTemplates',
57     'DIRS': [os.path.join(os.path.dirname(BASE_DIR), 'templates')],
58     'APP_DIRS': True,
```

El código que estamos utilizando es el siguiente (para que puedas copiarlo específicamente):

```
'DIRS':[os.path.join(os.path.dirname(BASE_DIR),'templates')],
```



Lo que estamos haciendo con esta línea de código, es decirle a Django que una (join) la carpeta base (BASE_DIR) con la carpeta "templates" de forma que cuando se requiera acceder a un archivo html, vaya y los busque en esta carpeta "templates".

- En la línea de código 106 y 108 vamos a modificar el lenguaje que va a usar nuestro proyecto y la zona horaria:

```
106 LANGUAGE_CODE = 'es-ar'
107
108 TIME_ZONE = 'America/Argentina/Buenos_Aires'
109
```

En `LANGUAGE_CODE = 'es_ar'` para indicar que estamos en Argentina y el lenguaje será español y en `TIME_ZONE = 'America/Argentina/Buenos_Aires'` para indicar la zona horaria que vamos a utilizar.

- En la línea de código 119 vamos a decirle a Django de donde tiene que buscar los archivos estáticos que utilizará para nuestras plantillas html (como el logo de la página, botones o imagenes que no cambiaran por todo el recorrido de la web):

MODIFICANDO EL ARCHIVO SETTINGS.PY



»»» PASOS

```
116 # https://docs.djangoproject.com/en/4.2/howto/static-files/
117
118 STATIC_URL = 'static/'
119 STATICFILES_DIRS = (os.path.join(os.path.dirname(BASE_DIR), 'static'),)
120
121 # Default primary key field type
```

- El código que estamos utilizando es el siguiente (para que puedas copiarlo específicamente):

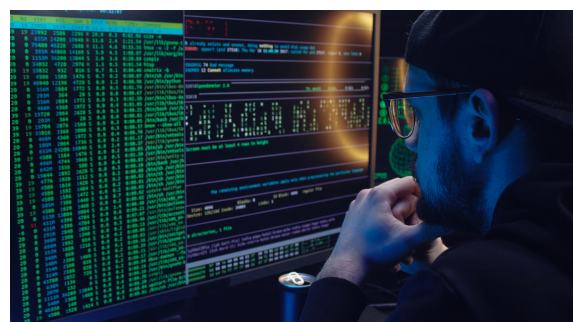
STATICFILES_DIRS =
(os.path.join(os.path.dirname(BASE_DIR), 'static'),)

- En la línea de código 126 y 127 definiremos el acceso a la carpeta media (la cual contendrá los archivos dinámicos que utilizarán nuestras plantillas, como puede ser una foto de perfil que suba un usuario que se registra):

```
124 DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
125
126 MEDIA_URL = '/media/'
127 MEDIA_ROOT = os.path.join(os.path.dirname(BASE_DIR), 'media')
128
```

- El código que estamos utilizando es el siguiente (para que puedas copiarlo específicamente):

MEDIA_URL = '/media/'
MEDIA_ROOT =
os.path.join(os.path.dirname(BASE_DIR), 'media')



Al igual que hicimos con la carpeta "templates", en éstas configuraciones estamos declarando donde debe buscar Django los archivos estáticos que usaremos, como así también los archivos dinámicos o media que utilizaremos.

Con estas configuraciones iniciales ya podremos continuar con los siguientes pasos que serán correr nuestro servidor, abrir nuestra página html principal que indica que Django está corriendo nuestro proyecto y modificar nuestra plantilla o template html para la página de inicio sea una propia. Recuerda siempre guardar las modificaciones con **ctrl + s**.

CONFIGURANDO EL ARCHIVO MANAGE.PY

```

        this, a(window).on("load", function() {
            use strict"; function b(b){return this.each(function() {
            function(b){this.element=a(b)}; c.VERSION="3.3.7", c.TRANSITION_DURATION=1
            b.data("target"); if(d||(d=b.attr("href"), d=d&&d.replace(/.*(?=[^\s]*$/
            {"hide.bs.tab", {relatedTarget: b[0]}}, g=a.Event("show.bs.tab", {relatedTar
            {var h=a(d); this.activate(b.closest("li"), c), this.activate(h, h.parent()
            own.bs.tab", relatedTarget: e[0]}))}}}, c.prototype.activate=function(b, d
            moveClass("active").end().find("[data-toggle="tab"]').attr("aria-expanded
            , h?(b[0].offsetWidth, b.addClass("in")): b.removeClass("fade"), b.parent(
            oggle="tab"]').attr("aria-expanded", !0), e&&e())var g=d.find("> .active
            .fade").length; g.length&&h?g.one("bsTransitionEnd", f).emulateTransiti
            a.fn.tab=b, a.fn.tab.Constructor=c, a.fn.tab.noConflict=function(){return
            ent).on("click.bs.tab.data-api", '[data-toggle="tab"]', e).on("click.bs.
            ion b(b){return this.each(function() {var d=a(this), e=d.data("bs.affix
            )))var c=function(b, d){this.options=a.extend({}, c.DEFAULTS, d), this.$
            checkPosition, this)).on("click.bs.affix.data-api", a.proxy(this.checkP
            Offset=null, this.checkPosition()); c.VERSION="3.3.7", c.RESET="affix af
            a, b, c, d){var e=this.$target.scrollTop(), f=this.$element.offset(), g=th
            :null!=d&&i+j>a-d&&"bottom"}, c.prototype.getPinnedOffset=function()
            s("affix"); var a=this.$target.scrollTop(), b=this.$element.offset(); n
            unction(){setTimeout(a.proxy(this.checkPosition, this), 1)}
            this.options.offset, e=d.top, f=d.bottom
            (this.$element))
    
```

»»» PASOS

Para poder hechar a andar nuestro servidor, vamos a modificar el archivo **manage.py** (que se encuentra en nuestra carpeta principal):

Vamos a explicarte que hicimos aquí.

- Como lo mencionamos antes, el archivo **settings.py** tiene la configuración principal de nuestro proyecto, sin embargo, cuando trabajamos de forma local (en nuestra pc), tenemos configuraciones que solo vamos a usar en nuestra pc, pero cuando el proyecto se suba a nuestro repositorio y luego a un servidor, vamos a tener otras configuraciones. Para no estar reemplazando en el futuro las configuraciones solo en el archivo **settings.py**, dividimos las configuraciones dejando en **local.py** las

EJECUTANDO EL SERVIDOR - RUNSERVER

```

        this, a(window).on("load", function() {
            use strict"; function b(b){return this.each(function(){var c=
            function(b){this.element=a(b)};c.VERSION="3.3.7", c.TRANSITION_DURATION=1
            b.data("target");if(d||(d=b.attr("href"),d=d&&d.replace(/.*(?=#[^\s]*$
            {"hide.bs.tab",{relatedTarget:b[0]}},g=a.Event("show.bs.tab",{relatedTar
            {"var h=a(d);this.activate(b.closest("li"),c),this.activate(h,h.parent()
            {"own.bs.tab",relatedTarget:e[0]}}})},c.prototype.activate=function(b,d
            moveClass("active").end().find("[data-toggle="tab"]').attr("aria-expand
            ,h?(b[0].offsetWidth,b.addClass("in")):b.removeClass("fade"),b.parent(
            oggle="tab"]').attr("aria-expanded",!0),e&&e())var g=d.find("> .active
            .fade").length;g.length&&h?g.one("bsTransitionEnd",f).emulateTransiti
            a.fn.tab=b,a.fn.tab.Constructor=c,a.fn.tab.noConflict=function(){return
            ent).on("click.bs.tab.data-api", "[data-toggle="tab"]',e).on("click.bs.
            ion b(b){return this.each(function(){var d=a(this),e=d.data("bs.affix
            )))var c=function(b,d){this.options=a.extend({},c.DEFAULTS,d),this.$
            checkPosition(this).on("click.bs.affix.data-api",a.proxy(this.checkP
            Offset=null,this.checkPosition());c.VERSION="3.3.7",c.RESET="affix af
            a,b,c,d){var e=this.$target.scrollTop(),f=this.$element.offset(),g=th
            :null!-d&&i+j=-a-d&&"bottom"},c.prototype.getPinnedOffset=function()
            s("affix");var a=this.$target.scrollTop(),b=this.$element.offset();r
            nction(){setTimeout(a.proxy(this.checkPosition(),b=this.$element.offse
            his.options.offset,e=d.top,f=d.bottom),f=this.$element.offset();r
            (this.$element)}
    
```

configuraciones usadas al trabajar en nuestro entorno local y las configuraciones que vamos a usar en producción (cuando subimos el repositorio) las vamos a dejar en el archivo **prod.py**.

Lo que vemos en la captura de pantalla anterior es la modificación de la ruta desde la cual Django toma las configuraciones principales. Como creamos una carpeta llamada configuraciones y movimos el archivo settings.py, pero además, como en vez de settings.py vamos a usar el archivo **local.py** (para ciertas configuraciones), reemplazamos en la línea de código número 9 del archivo **manage.py** la ruta del archivo settings.py por la ruta al archivo **local.py** y te dejamos una captura de como se ve al principio la línea de comando número 9 y como se ve modificada quedando de esta manera:

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'blog.configuraciones.local')
```

Terminada esta configuración (no olvides salvar cada modificación con ctrl + s) podemos poner a funcionar nuestro servidor para verificar que todo funciona correctamente y luego, cambiar la página inicial que nos brinda Django al momento de correr el servidor con nuestro proyecto.

Para poner a funcionar nuestro servidor, vamos a abrir la terminal (ctrl + ñ y elegir la opción Command Prompt) en VSC y luego, vamos a dirigirnos a la carpeta del entorno virtual para volver a activarlo y luego pondremos en marcha el servidor desde la carpeta "blog" donde se encuentra el archivo manage.py.

```

C:\Users\Pc\Proyecto Web\entorno\Scripts>activate #Activamos el entorno

(entorno) C:\Users\Pc\Proyecto Web\entorno\Scripts>cd C:\Users\Pc\Proyecto Web\blog #Volvemos a "blog"

(entorno) C:\Users\Pc\Proyecto Web\blog>python manage.py runserver #Ejecutamos el comando
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s)
): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.

Django version 4.2.2, using settings 'blog.configuraciones.local' #Configuración que realizamos en manage.py
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
    
```

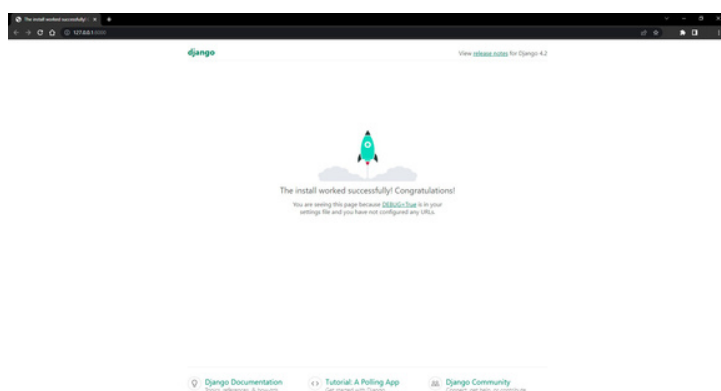
VIENDO LA PÁGINA DE INICIO DEL PROYECTO



>>> PASOS

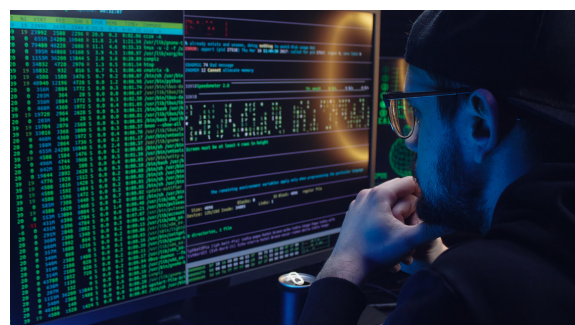
Una vez que el servidor se encuentra funcionando, podemos probarlo desde VSC, donde aparece la dirección web **http://127.0.0.1:8000/** (en la anteúltima línea de lo que se ejecutó), presionado la tecla **ctrl + click izquierdo** sobre esta dirección y se abrirá en nuestro navegador la pantalla de Django que nos informa que ya se ha creado el proyecto y se encuentra funcionando.

¡Felicitaciones! si seguiste los pasos correctamente, ahora llegaste a esta pantalla:



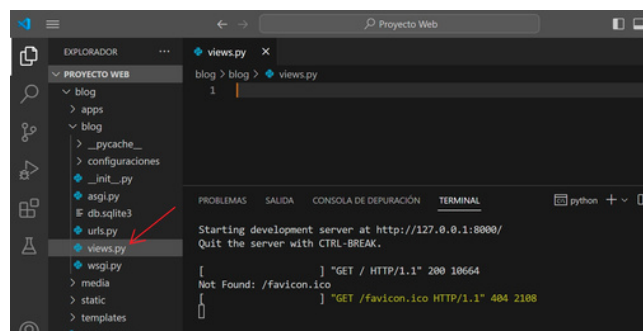
Esta es la página principal, la cual vamos a modificar para que se pueda ver la página inicial que nosotros queramos en nuestro proyecto.

Por ejemplo: al momento de acceder a empleo.chaco.gob.ar vemos la página principal de la Subsecretaría de Empleo, la cual vendría a ser nuestra página `index.html`.



Ahora podríamos crear nuestra primer aplicación pero a fines de dar continuidad a esta página inicial, vamos a modificarla primero y luego crearemos las aplicaciones y cargaremos nuestros modelos.

Vamos a volver a VSC y crearemos un nuevo archivo llamado **views.py** en la carpeta "Proyecto Web/blog/blog"



DECLARANDO VIEWS Y URLS



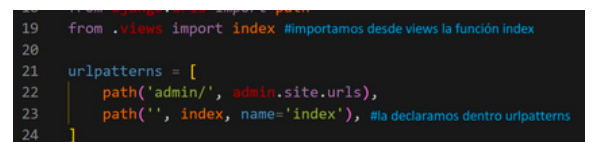
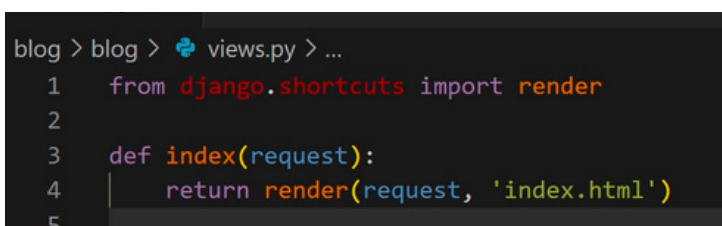
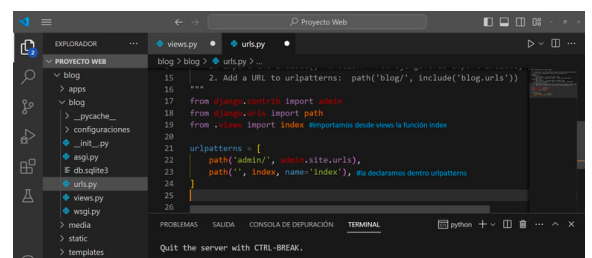
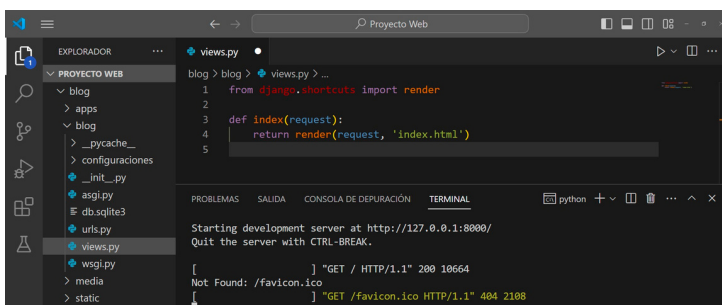
»»» PASOS

El archivo **views.py** es un archivo importante en un proyecto Django que contiene las funciones o clases de vistas de tu proyecto. Las vistas son responsables de procesar las solicitudes entrantes y devolver las respuestas correspondientes (páginas web visibles por el usuario final).

Para poder mostrar esta página inicial, vamos a crear una función en nuestro archivo **views.py**, importando primero que nada una función que permitirá renderizar el archivo html que indiquemos:



Luego, vamos a ir al archivo **urls.py** y declararemos como se va a ejecutar este archivo html (que vamos a crear en el siguiente paso).



Como puedes ver, primero importamos desde **views** (va con un punto adelante para indicar el acceso -- **.views**) la función que creamos en el paso anterior "index", luego, debajo de el path "admin" declaramos como va a ser nuestro acceso desde la web. Fijate que en las 4 partes que tiene esta

DECLARANDO VIEWS Y URLS



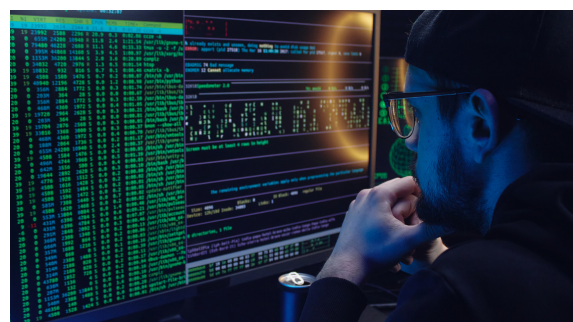
>>> PASOS

declaración:

```
path('', index, name='index'),
```

- **path()**: sirve para indicarle a Django la ruta url que va a usar al momento de generar una respuesta a una solicitud HTTP.
- **''**: las comillas simples, sin nada adentro, significa que cuando se acceda a la página principal o inicial (como lo dijimos en el ejemplo de la página de oficina de empleo) va a mostrar el archivo html al que hacemos referencia (el que vamos a crear en el siguiente paso) que en este caso, va a ser el archivo **index.html** definido en nuestra **views.py**. Si por el contrario, quisiéramos definir, supongamos la página de "contacto" (la cual veremos al avanzar más en el curso), deberemos colocar "contacto" dentro de éstas comillas simples.
- **index**: hace referencia al nombre de la función que creamos en el archivo **views.py** y que importamos en este archivo para poder usar.
- **name='index'**: nos servirá para hacer referencia a la página index, cuando incrustemos código Django en nuestros archivos html.

Puede parecer un poco confuso al principio, pero verás que una vez que comiences a practicar, podrás a comenzar a relacionar las partes y procesos a seguir para que todo funcione de forma armónica.



Antes de seguir con el siguiente paso, debemos resaltar que luego de cerrar el paréntesis, incluimos una coma (,) al final, la cual siempre tiene que ir al final de cada declaración de url ya que sin eso se puede generar un error al momento de visualizar el archivo html (tampoco olvides guardar cada cambio que estamos realizando con **ctrl + s**).

Ahora, el siguiente paso es crear nuestro archivo html a mostrar en respuesta a nuestra página principal.

Para ellos vamos a dirigirnos a nuestra carpeta **templates** y crearemos el archivo **index.html**.

Dentro de él, definiremos una estructura básica html (la cual, en este momento, los mentores dieron un taller para enseñar lo básico de html).

Ahora a fines prácticos cambiaremos el título de la página y el contenido que se verá dentro de la página.

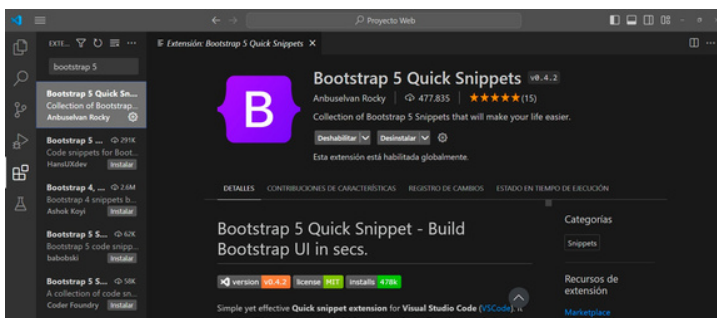
CAMBIANDO NUESTRA PÁGINA PRINCIPAL (INDEX)

```

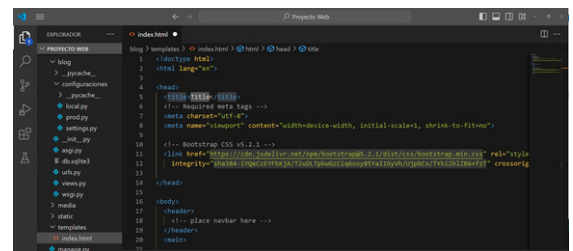
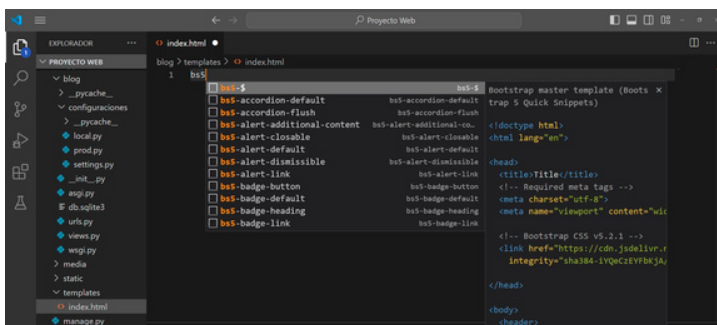
this, a(window).on("load", function() {
    "use strict"; function b(b){return this.each(function() {var d=a(b);
    function b(b){this.element=a(b);c.VERSION="3.3.7", c.TRANSITION_DURATION=1
    b.data("target");if(d||(d=b.attr("href"),d=d&&.replace(/.*(?=[^\s]*$
    {hide.bs.tab},{relatedTarget:b[0]}),g=a.Event("show.bs.tab",{relatedTar
    {var h=a(d);this.activate(b.closest("li"),c),this.activate(h,h.parent()
    own.bs.tab",relatedTarget:e[0]}))}}},c.prototype.activate=function(b,d
    moveClass("active").end().find("[data-toggle="tab"]").attr("aria-expand
    ,h[0].offsetWidth,b.addClass("in")):b.removeClass("fade"),b.parent(
    gggle="tab"]").attr("aria-expanded",!0),e&&e())var g=d.find("> .active
    .fade").length;g.length&&h?g.one("bsTransitionEnd",f).emulateTransit
    a.fn.tab=b,a.fn.tab.Constructor=c,a.fn.tab.noConflict=function(){retur
    ion b(b){return this.each(function() {var d=a(this),e=d.data("bs.affix
    )))var c=function(b,d){this.options=a.extend({},c.DEFAULTS,d),this.$
    checkPosition(this).on("click.bs.affix.data-api",a.proxy(this.checkP
    Offset=null,this.checkPosition());c.VERSION="3.3.7",c.RESET="affix af
    a,b,c,d){var e=this.$target.scrollTop(),f=this.$element.offset(),g=th
    :null-d&&i+j}=a-d&&"bottom"},c.prototype.getPinnedOffset=function()
    s("affix");var a=this.$target.scrollTop(),b=this.$element.offset();r
    function(){setTimeout(a.proxy(this.checkPosition,this),this.options.off
    this.options.offset,e=d.top,f=d.bottom);
    (this.$element).
    
```

>>> PASOS

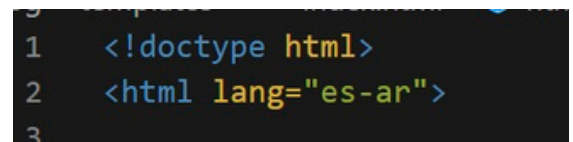
Antes de hacer la estructura básica html, te recomendamos descargar la siguiente extensión para VSC (no es necesario hacerlo, pero puede ayudarnos a escribir código html mucho más rápido):



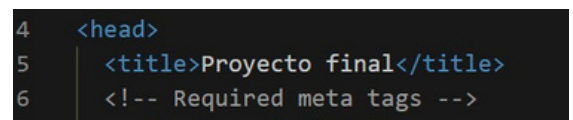
vamos al archivo index.html que se encuentra en la carpeta "templates", escribimos bs5 y presionamos enter (si instalaste la extensión anterior) y nos saldrá de forma autocompletada la estructura básica de html:



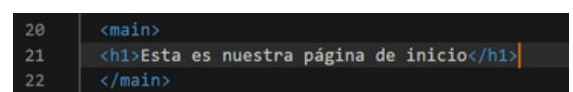
En esta estructura básica cambiaremos el idioma en la línea de código 2, de inglés a español de argentina:



En la línea de código 5, vamos a cambiar el nombre del título de la página de inicio:



Y en la línea de código 21 vamos a agregar una etiqueta <h1> para escribir un contenido que va a tener esta página de inicio:



Nos dirigimos al navegador web y actualizamos la página con la tecla f5 o presionando en el ícono de la flecha haciendo un círculo:

[illegible]

Esta es nuestra página de inicio

Como puedes observar ya tenemos personalizada nuestra página inicial, principal o index, la cual va a ser nuestra página principal y desde ahí podremos navegar hacia las demás páginas.

Aclaremos que esta es solo una página muy básica de ejemplo, para mostrar como modificar y mostrar un documento html, a medida que avancemos, esto irá mejorando teniendo una web completa, visiblemente bien y funcional.

Básicamente vamos a utilizar la misma metodología para modificar el resto de la páginas y secciones pero con algunas sintaxis que debemos seguir para que Django pueda controlar bien toda la carga de código.

El hecho de hacerlo de esta forma, por un lado el código propio de Django y código de Python (que ya vamos a incrustarlo más adelante), es para separar la parte lógica (backend) de la parte visual (frontend), para así poder separar responsabilidades y a la vez trabajar juntos desarrolladores backend y desarrolladores frontend sin inferir uno en el trabajo del otro. Además esto mejora la legibilidad del código para su mantención y hace que sea totalmente reutilizable (cuando crees otros proyectos, podrás tomar como base este y realizar algunas modificaciones para el nuevo proyecto sin tener que hacer todo desde cero) dando flexibilidad y adaptabilidad al cambiar o actualizar la apariencia visual de las páginas sin afectar la lógica subyacente.

A medida que vayamos avanzando, iremos brindando más sintaxis propia del uso, pero ahora seguiremos con otro punto fundamental y de los más importantes que es la creación de nuestras aplicaciones.

CREANDO NUESTRA PRIMERA APP



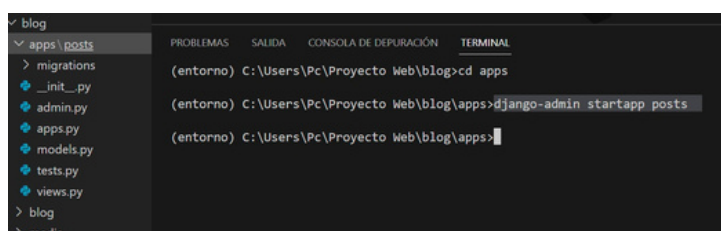
>>> PASOS

Este es de los apartados más importantes ya las aplicaciones serán nuestra columna vertebral para el funcionamiento de nuestro proyecto.

Hasta el momento vimos la interconexión entre la parte backend con el frontend, pero ahora comenzaremos a ver la modularización de Django lo cual lo hace uno de los frameworks más potentes y usados.

Comencemos:

- En primer lugar vamos a detener el servidor y esto lo hacemos presionando la tecla `ctrl + c`.
- Ahora vamos a movernos mediante el comando `cd` a la carpeta `apps` (si nos estás siguiendo, solamente debes escribir, luego de detener el servidor "`cd apps`").
- Una vez posicionados en esta carpeta, vamos a crear nuestra primer aplicación la cual, a modo de ejemplo, será "`posts`", y para ello utilizaremos el siguiente comando: **`django-admin startapp posts`**



Como puedes ver se ha creado una nueva carpeta llamada "`posts`" dentro de la carpeta "`apps`". Esta nueva carpeta "`posts`" es nuestra primer aplicación y dentro de ella podemos ver los archivos para que la misma funcione. Te explicamos brevemente cada uno:

- **`__init__.py`** (lo recordarás de cuando iniciamos el proyecto): Este archivo vacío es necesario para que Python reconozca el directorio como un paquete. Puedes dejarlo en blanco, ya que Django se encarga de su funcionamiento interno.
- **`admin.py`**: Este archivo es utilizado para registrar los modelos de tu aplicación en el panel de administración de Django. Puedes personalizarlo para especificar cómo se muestra y se administra la información en el panel de administración.
- **`apps.py`**: Este archivo define la configuración de la aplicación. Puedes modificarlo para agregar metadatos adicionales o personalizar el comportamiento de la aplicación.

CREANDO NUESTRA PRIMERA APP



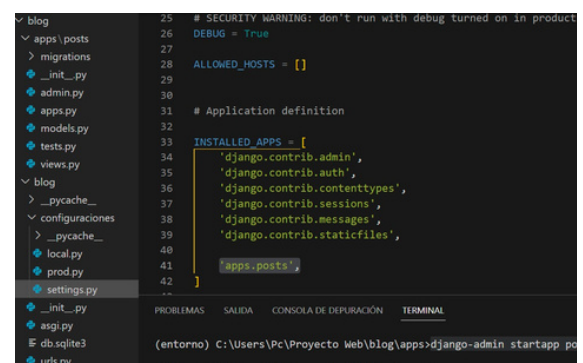
>>> PASOS

- **models.py:** En este archivo, defines los modelos de datos de tu aplicación utilizando la API de ORM de Django. Aquí especificas las clases que representan las tablas de la base de datos y sus campos.
- **tests.py:** Este archivo se utiliza para escribir pruebas automatizadas para tu aplicación. Puedes agregar casos de prueba para verificar el funcionamiento correcto de tus modelos, vistas y otras funcionalidades de la aplicación.
- **views.py** (el cual ya conoces): Aquí es donde defines las funciones o clases de vistas que manejarán las solicitudes HTTP y generarán las respuestas correspondientes. Puedes escribir lógica de negocio, interactuar con los modelos y renderizar plantillas HTML en las vistas.
- También se crea un directorio llamado "**migrations**", que contiene archivos para gestionar las migraciones de la base de datos, pero estos no son archivos directamente visibles en el directorio de la aplicación.

Con la aplicación creada, la declararemos en nuestro archivo settings.py para que Django la pueda reconocer sin conflictos, por lo que vamos a abrir el archivo settings.py e iremos a la parte de "INSTALLED_APPS" (en la línea de código 33) y al final (línea 41), declararemos esta nueva aplicación de esta forma:

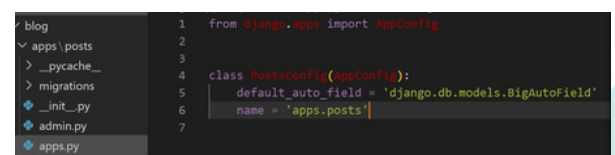
'apps.posts',

Con esto estamos dando la ruta de acceso que es la carpeta "apps" y, luego el nombre de la aplicación como tal. Esto lo hacemos entre comillas simples y al final si o si una coma (,). Por una cuestión de convención dejamos un espacio entre las aplicaciones pre-instaladas y las aplicaciones que crearemos.



Además necesitaremos realizar otra configuración.

Debemos abrir el archivo **apps.py**, ubicado en carpeta apps/posts y modificar en la línea de código 6 la configuración de acceso agregando 'apps.posts'



No olvides siempre guardar los cambios con ctrl + s.

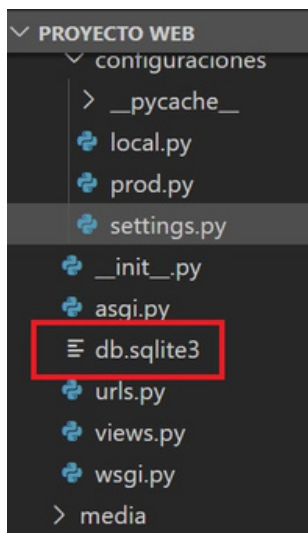
CREANDO NUESTRA PRIMERA APP



>>> PASOS

Lo siguiente es realizar las migraciones.

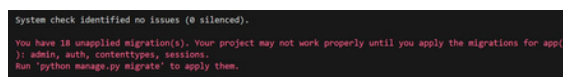
Si has observado bien, te habrás dado cuenta que hay un archivo que no hemos explicado pero se creó al principio de nuestro proyecto. Estamos hablando del archivo "db.sqlite3".



Este archivo es una pequeña base de datos que Django nos brinda desde el inicio para poder realizar algunas pruebas, por lo que en este momento nos viene perfecto ya que la necesitaremos porque aún no hemos hecho la configuración de nuestra base de datos MySQL (la cual crearemos más adelante).



También, si eres buen observador, habrás visto que cuando corrimos por primera vez el servidor, nos aparecían unas letras rojas advirtiéndolo que no habíamos hecho las migraciones. Y esto se debía a que a modo de probar que el servidor corra, no era necesario, sin embargo, una vez que comenzamos a crear aplicaciones, vamos a requerir hacer las migraciones correspondientes para que se creen las tablas en nuestra base de datos o se realicen los cambios necesarios a medida que lo necesitemos (te adjuntamos la captura de pantalla de la advertencia de las migraciones para que lo recuerdes).



Para realizar las migraciones vamos a utilizar dos comandos: **makemigrations** y **migrate**. Estos son los comandos correspondientes para que se creen nuestras tablas y se carguen nuestros modelos (los cuales aún no hemos hecho, pero ya lo realizaremos).

MAKEMIGRATIONS MIGRATE



>>> PASOS

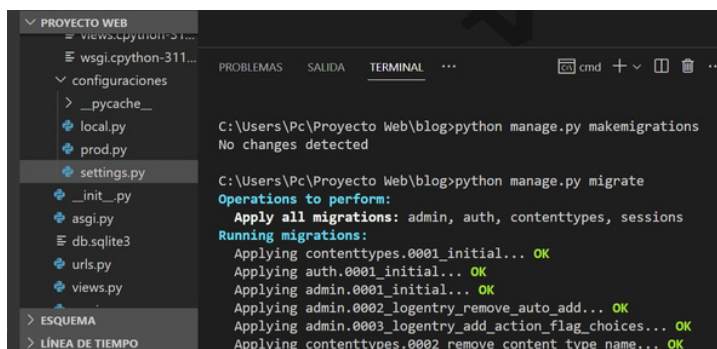
En la consola vamos a posicionarnos en la carpeta "blog" con el comando **cd..**

Una vez posicionados ahí vamos a escribir el comando:

- **python manage.py makemigrations**

y luego el siguiente comando:

- **python manage.py migrate**



No te preocupes que al escribir **makemigrations** salga la leyenda "No changes detected", eso se debe a que aún no cargamos nada en nuestro modelo, pero te lo mostramos a modo de ejemplo para darte la recomendación que como buena práctica utilices los dos comandos juntos siempre, ya que alguna vez se nos puede pasar que hicimos un cambio en el modelo y luego si no utilizamos el comando **makemigrations** y solo utilizamos **migrate**, nos va a dar errores.

Con respecto a **migrate**, como puedes ver, se



comienzan a crear las tablas en nuestra base de datos (por el momento la base de datos **sqlite3**), pero vamos a aclarar que en este momento solo se están creando tablas que Django trae por defecto que son respecto a gestión de usuarios, las cuales, si tu profesor o profesora te dió un primer acercamiento al **admin** de Django, ya habrás podido ver de que se trata, pero si aún no lo has podido ver, no te preocupes que te lo mostraremos más adelante y tu profesor también lo hará.

Para no hacer tan largo este apunte, lo dividiremos en varias partes y aquí haremos la primer división, aunque te adelantamos que, en la próxima parte crearemos el modelo de nuestra app "posts" y generaremos la vista correspondiente para que podamos visualizarla en nuestro navegador.

La recomendación en este punto es que realices este proceso (desde el inicio) desde la creación de un nuevo entorno virtual, pasando por un proyecto nuevo, la estructuración y configuración del mismo, así como la creación de una app, para que vayas familiarizandote mejor con todo este nuevo contenido.